



南開大學
Nankai University

《并行与分布式程序设计》实验报告

(2025~2026 学年第一学期)

实验名称: Pthread 编程

姓 名: 李宗杭

学 号: 2311451

专 业: 软件工程

2025 年 11 月 23 日

目 录

1	“多个数组排序”的任务不均衡案例复现与优化	2
1.1	问题描述	2
1.2	算法设计与实现	2
1.2.1	数组初始化	2
1.2.2	静态分配任务	2
1.2.3	动态分配任务	3
1.2.4	动态分配任务-粗粒度	4
1.3	实验及结果分析	4
1.3.1	测试方法	4
1.3.2	结果分析	4
1.3.3	实际运行结果	8
2	高斯消元法 Pthread 多线程实现	10
2.1	问题描述	10
2.2	算法设计与实现	10
2.2.1	动态分配线程	10
2.2.2	静态任务（动态分配线程）	11
2.2.3	动态任务（动态分配线程）	11
2.2.4	静态分配线程	12
2.2.5	静态任务（静态分配线程）	13
2.2.6	动态任务（静态分配线程）	13
2.3	实验及结果分析	14
2.3.1	测试方法	14
2.3.2	结果分析	14
2.3.3	实际运行结果	21
3	Barrier 功能多种实现与对比	23
3.1	问题描述	23
3.2	算法设计与实现	23
3.2.1	条件变量 + 互斥量	23
3.2.2	信号量	24
3.2.3	测试函数	25
3.3	实验及结果分析	27
3.3.1	测试方法	27
3.3.2	结果分析	27
3.3.3	实际运行结果	29

1 “多个数组排序”的任务不均衡案例复现与优化

1.1 问题描述

复现课件中“多个数组排序”的任务不均衡场景，通过调整任务分块大小、线程数、静态/动态任务分配策略，探索最优解决方案，确保多线程负载均衡。

1.2 算法设计与实现

1.2.1 数组初始化

人为控制生成规律性的数组数据，使得负载不均情况的出现。

```
1 void init_arr() {
2     seg = arr_num / thread_num;
3     arr.resize(arr_num);
4     srand(unsigned(time(NULL)));
5     int ratio; // 比例
6     for (int i = 0; i < arr_num; i++) {
7         arr[i].resize(arr_len);
8         // 分四段设置不同有序度
9         if (i < seg) ratio = 0; // 完全升序
10        else if (i < seg * 2) ratio = 32; // 1/4 升序
11        else if (i < seg * 3) ratio = 64; // 1/2 升序
12        else ratio = 128; // 完全逆序
13        if ((rand() & 127) < ratio) {
14            for (int j = 0; j < arr_len; j++) // 升序数组
15                arr[i][j] = j;
16        } else {
17            for (int j = 0; j < arr_len; j++) // 降序数组
18                arr[i][j] = arr_len - j;
19        }
20    }
21 }
```

1.2.2 静态分配任务

在创建线程之前便分配好每个线程负责的任务，按照线程数量均分。

```
1 // 静态划分（负载不均衡）
2 void *arr_sort_static(void *parm) {
3     int r = *(int *)parm;
4     // 每个线程处理 seg 个数组
5     for (int i = r * seg; i < (r + 1) * seg; i++) {
6         stable_sort(arr[i].begin(), arr[i].end());
7     }
```

```

7     }
8     pthread_exit(NULL);
9     return NULL;
10 }
11
12 // 测试静态划分
13 double test_static() {
14     seg = arr_num / thread_num;
15     pthread_t threads[thread_num];
16     int id[thread_num];
17     LARGE_INTEGER head, tail;
18     QueryPerformanceCounter(&head);
19     for (int i = 0; i < thread_num; i++) {
20         id[i] = i;
21         pthread_create(&threads[i], NULL, arr_sort_static, &id[i]);
22     }
23     for (int i = 0; i < thread_num; i++) {
24         pthread_join(threads[i], NULL);
25     }
26     QueryPerformanceCounter(&tail);
27     double during = (tail.QuadPart - head.QuadPart) * 1000.0 / freq.QuadPart;
28     return during;
29 }

```

后续动态分配任务和粗粒度动态的测试函数与 test_static() 基本一致，仅多了互斥量的初始化与销毁以及创建线程传递的线程函数不一样，不再重复展示。

1.2.3 动态分配任务

使用全局变量 next_row 来指定下一个待处理的行，线程间互斥访问这个变量来动态获取自己即将处理的行并更新变量的值。

```

1 // 动态划分（负载均衡）
2 void *arr_sort_dynamic(void *parm) {
3     int r = *(int *)parm;
4     int task;
5     while (1) {
6         pthread_mutex_lock(&mutex_task);
7         task = next_row++;
8         pthread_mutex_unlock(&mutex_task);
9         if (task >= arr_num) break;
10        stable_sort(arr[task].begin(), arr[task].end());
11    }

```

```
12     pthread_exit(NULL);
13     return NULL;
14 }
```

1.2.4 动态分配任务-粗粒度

与细粒度代码基本一致，仅在更新 `next_row` 时增量由 1 变为 `block_size`。

```
1 // 动态划分（每次分配 block_size 个任务）
2 void *arr_sort_coarse(void *parm) {
3     int r = *(int *)parm;
4     int task_start;
5     while (1) {
6         pthread_mutex_lock(&mutex_task);
7         task_start = next_row;
8         next_row += block_size;
9         pthread_mutex_unlock(&mutex_task);
10        if (task_start >= arr_num) break;
11        int task_end = min(task_start + block_size, arr_num);
12        for (int task = task_start; task < task_end; task++) {
13            stable_sort(arr[task].begin(), arr[task].end());
14        }
15    }
16    pthread_exit(NULL);
17    return NULL;
18 }
```

1.3 实验及结果分析

1.3.1 测试方法

首先取一个较大的数据规模，并为粗粒度动态方法设置一个适当的粒度，然后对不同的线程数的情况进行测试。

随后选取一个较优的线程数，在此规模下，再次对不同粒度的情况进行测试。

最后测试选取的较优线程数和较优粒度在不同规模数据下的表现。

1.3.2 结果分析

(1) 不同线程数下三种划分策略的运行时间对比

首先分析线程数对性能的影响，测试时数据规模为 10000，粗粒度动态划分粒度为 50。整体来看，随着线程数的增加，三种划分方式的耗时均呈下降趋势，但下降速率逐渐放缓，在线程数来到 18 时，三种方式耗时均基本稳定。具体来看，耗时稳定后，静态划分维持在 540ms 左右，动态划分在 500ms 左右，粗粒度动态划分在 510ms 左右。

静态划分按连续块分配任务，数组有序度的差异导致了负载不均。而动态划分通过抢占式获取任务，平衡了各线程工作量，充分利用多核资源。

目前来看，在这种情况下（数据规模 10000，粒度 50），效率：动态划分 > 粗粒动态 > 静态划分。

线程数	静态划分	动态划分	动态粗粒
2	2787.5	2726.1	2734.4
4	1475	1327.7	1336
6	1015.8	952.29	968
8	881.69	764.38	778.1
10	755.51	658.08	673.21
12	672.87	590.77	608.19
14	641.55	560.12	569.66
16	572.03	498.4	516.65
18	534.48	480.51	492.49
20	506.33	450.1	458.96
22	487.58	425.07	435.64
24	470.44	403.76	421.18
26	458.95	414.62	416.99
28	434.86	380.76	403.37
30	439.78	383.12	397.47
32	433.56	379.75	407.01

表 1: 不同线程数下三种划分策略的运行时间（数据规模：10000，分配粒度：50，单位：ms）

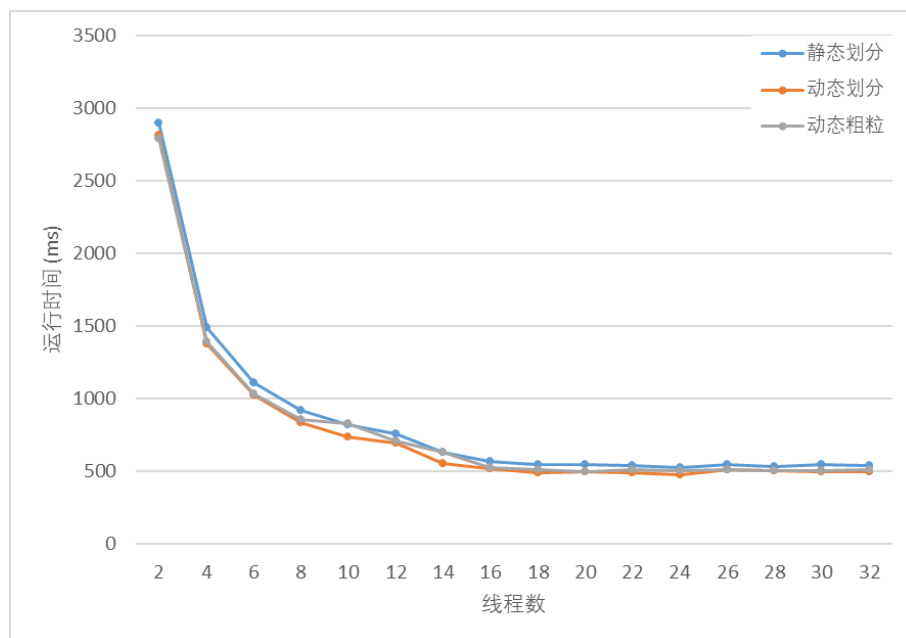


图 1.1: 不同线程数下三种划分策略的运行时间（数据规模：10000，分配粒度：50）

(2) 不同分配粒度下三种划分策略的运行时间对比

根据上面测试的结果，选用 32 线程，保持 10000 的数据规模，继续对不同粒度的情况进行测试。粒度对静态划分和普通动态划分无影响，动态划分耗时稳定优于静态划分。粗粒度动态划分的数

据波动较大，但整体而言耗时呈现上升趋势，从一开始的介于静态划分和动态划分之间，到最后甚至超过了静态划分。

分配任务的粒度大虽然能减少分配任务的次数，但是总的运行时间不一定好。分配任务大的时候，线程之间运行时间方差会越来越大，会导致总的运行时间被拖累，重新出现负载不均问题。从图1.2可知，最好的效果还是细粒度每次分配 1 行进行计算。

分配粒度	静态划分	动态划分	动态粗粒
5	533.76	514.12	490.61
10	517.52	509.50	499.85
15	517.14	506.64	492.62
20	516.64	481.85	495.02
25	523.86	500.49	508.99
50	510.08	476.05	492.14
75	517.16	482.79	496.22
100	516.61	485.65	528.55
125	507.16	488.36	504.16
150	514.01	492.76	507.03
175	518.27	484.78	536.92
200	521.97	490.91	591.55
225	516.65	500.78	515.02
250	519.79	486.60	499.56
275	521.86	481.87	507.17
300	530.81	495.87	515.78
325	520.83	483.63	514.82
350	509.68	478.87	516.38
375	507.86	485.12	518.84
400	510.84	477.75	518.36
425	506.59	480.35	537.00
450	502.42	482.46	550.47
475	517.40	480.47	549.52
500	525.80	490.76	562.28

表 2: 不同分配粒度下三种划分策略的运行时间（数据规模：10000，线程数：32，单位：ms）

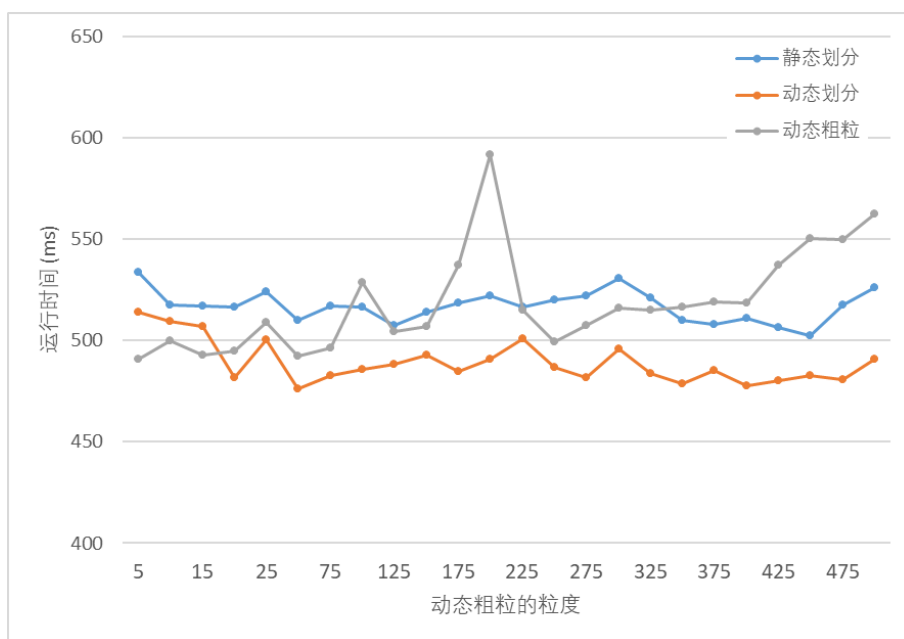


图 1.2: 不同分配粒度下三种划分策略的运行时间 (数据规模: 10000, 线程数: 32)

(3) 不同数据规模下三种划分策略的运行时间对比

最后再采用较优的线程数 32 和粒度 50 在不同规模的数据情况下进行测试。

随着数据规模的增长, 三种划分方式的耗时均近似线性增长, 无明显非线性波动。细粒度与粗粒度动态划分几乎无差异, 均优于静态划分, 且随着规模的增大, 动态划分的优势也逐渐变大, 与静态划分的耗时差距越来越大。

虽然图中细粒度动态和粗粒度动态几乎不分上下, 但从表3的具体数据中还是可以看出, 细粒度动态划分始终略优于粗粒度动态划分。

数据规模	静态划分	动态划分	动态粗粒
5000	255.36	240.60	267.83
10000	529.62	498.54	499.90
15000	787.20	735.13	750.17
20000	1065.4	973.95	980.03
25000	1262.1	1226.1	1231.7
30000	1554.5	1433.6	1478.9
35000	1814.6	1672.6	1680.2
40000	2069.0	1910.8	1920.3
45000	2285.5	2155.8	2177.8
50000	2651.5	2493.1	2410.0

表 3: 不同数据规模下三种划分策略的运行时间 (分配粒度: 50, 线程数: 32, 单位: ms)

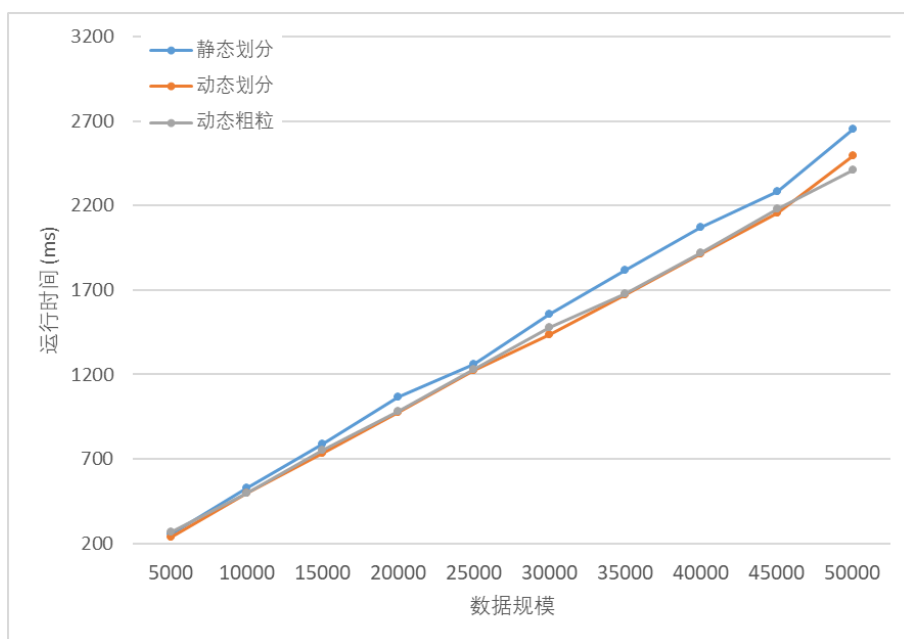


图 1.3: 不同数据规模下三种划分策略的运行时间 (分配粒度: 50, 线程数: 32)

1.3.3 实际运行结果

数据规模: 10000	分配粒度: 50			
线程数	静态划分	动态划分	动态粗粒	原总用时
2	2896.6	2814.6	2790.6	85018
4	1489.5	1379.1	1394.2	42629
6	1106.5	1028.1	1029.8	31644
8	919.95	837.42	853.41	26108
10	822.86	738.6	829.21	23907
12	754.76	693.48	706.21	21544
14	630.54	554.84	630.5	18159
16	570.68	517.98	527.17	16158
18	546.89	493.25	510.15	15503
20	548.23	496.32	500.12	15447
22	536.88	491.9	509.2	15380
24	529.53	476.48	501.98	15080
26	546.8	511.57	513.18	15716
28	532.3	505.72	508.23	15463
30	544.49	498.79	504.09	15474
32	540.06	500.1	510.29	15505

图 1.4: 不同线程数下三种划分策略的运行时间

数据规模: 10000		线程数: 32		
分配粒度	静态划分	动态划分	动态粗粒	原总用时
5	533.76	514.12	490.61	15385
10	517.52	509.5	499.85	15269
15	517.14	506.64	492.62	15164
20	516.64	481.85	495.02	14935
25	523.86	500.49	508.99	15333
50	510.08	476.05	492.14	14783
75	517.16	482.79	496.22	14962
100	516.61	485.65	528.55	15308
125	507.16	488.36	504.16	14997
150	514.01	492.76	507.03	15138
175	518.27	484.78	536.92	15400
200	521.97	490.91	591.55	16044
225	516.65	500.78	515.02	15324
250	519.79	486.6	499.56	15060
275	521.86	481.87	507.17	15109
300	530.81	495.87	515.78	15425
325	520.83	483.63	514.82	15193
350	509.68	478.87	516.38	15049
375	507.86	485.12	518.84	15118
400	510.84	477.75	518.36	15069
425	506.59	480.35	537	15239
450	502.42	482.46	550.47	15354
475	517.4	480.47	549.52	15474
500	525.8	490.76	562.28	15789

图 1.5: 不同分配粒度下三种划分策略的运行时间

分配粒度: 50		线程数: 32		
数据规模	静态划分	动态划分	动态粗粒	原总用时
5000	255.36	240.6	267.83	7637.9
10000	529.62	498.54	499.9	15281
15000	787.2	735.13	750.17	22725
20000	1065.4	973.95	980.03	30193
25000	1262.1	1226.1	1231.7	37199
30000	1554.5	1433.6	1478.9	44670
35000	1814.6	1672.6	1680.2	51674
40000	2069	1910.8	1920.3	59001
45000	2285.5	2155.8	2177.8	66192
50000	2651.5	2493.1	2410	75545

图 1.6: 不同数据规模下三种划分策略的运行时间

2 高斯消元法 Pthread 多线程实现

2.1 问题描述

实现高斯消去法解线性方程组的 Pthread 多线程编程，可与 SSE/AVX 编程结合，并探索优化任务分配方法。

2.2 算法设计与实现

本次实验共实现了八种并行策略：动态分配线程 + 静态分配任务、动态分配线程 + 动态分配任务、静态分配线程 + 静态分配任务、静态分配线程 + 动态分配任务，以及他们的 AVX 优化，且可通过全局变量 `chunk_size` 控制动态分配任务的粒度。同时实现了串行 AVX 策略作为对照。

实验仅对消元过程进行测试，不考虑回代求解。因 AVX 优化非本次实验重点，且其代码繁琐，报告中仅展示非 AVX 优化版本代码。

2.2.1 动态分配线程

采用这个函数来创建线程任务，暂且称这个函数为主进程。动态分配线程即在主进程中对主元行 `k` 进行迭代，每一次迭代中动态创建线程任务。静态任务负责的分块大小也在此静态设置为待处理行数与线程数的比值。

```

1 // 动态分配线程
2 void dynamic_thread(void* (*task)(void*)) {
3     vector<pthread_t> thread_handles(thread_num);
4     vector<threadParam_t> params(thread_num);
5     pthread_mutex_init(&mutex_task, NULL);
6     for (int k = 0; k < n; k++) {
7         if (task == dynamic_thread_static_simple_task || task ==
8             ↪ dynamic_thread_static_AVX_task) {
9             chunk_size = (n - (k + 1)) / thread_num + 1;    // 静态分配每个线程
10             ↪ 需要处理的行数
11         }
12         next_row = 0;
13         for (int t_id = 0; t_id < thread_num; t_id++) {
14             params[t_id].k = k;
15             params[t_id].t_id = t_id;
16         }
17         for (int t_id = 0; t_id < thread_num; t_id++) {
18             pthread_create(&thread_handles[t_id], NULL, task, &params[t_id]);
19         }
20         for (int t_id = 0; t_id < thread_num; t_id++) {
21             pthread_join(thread_handles[t_id], NULL);
22         }
23     }
24 }
```

```

22     pthread_mutex_destroy(&mutex_task);
23 }

```

2.2.2 静态任务（动态分配线程）

直接根据主进程传来的主元行 k 和分块大小进行处理，处理完该任务后结束线程。

```

1 // 静态任务
2 void* dynamic_thread_static_simple_task(void* param) {
3     threadParam_t* p = (threadParam_t*)param;
4     int k = p->k;
5     int t_id = p->t_id;
6     int start = k + t_id * chunk_size + 1;
7
8     for(int i = start; (i < start + chunk_size) && (i < n); i++) { // 消元
9         ↪ 操作，只处理分配的行块
10         float factor = A[i][k] / A[k][k];
11         for(int j = k; j < n; j++) {
12             A[i][j] -= factor * A[k][j];
13         }
14         B[i] -= factor * B[k];
15     }
16
17     pthread_exit(NULL);
18     return NULL;
19 }

```

2.2.3 动态任务（动态分配线程）

生命周期为主进程的一轮迭代，即对一个主元行 k 的处理。线程任务间通过对于 `next_row` 的互斥访问获取自己将要处理的行，随后对 `next_row` 进行更新，更新方式为 `next_row += chunk_size`，因此可通过调整 `chunk_size` 的大小来控制动态任务的粒度。

```

1 // 动态任务
2 void* dynamic_thread_dynamic_simple_task(void* param) {
3     threadParam_t* p = (threadParam_t*)param;
4     int k = p->k;
5     int t_id = p->t_id;
6     int start;
7
8     while (1) {
9         pthread_mutex_lock(&mutex_task);
10        start = k + next_row + 1;

```

```

11     next_row += chunk_size;
12     pthread_mutex_unlock(&mutex_task);
13
14     for(int i = start; (i < start + chunk_size) && (i < n); i++) {
15         float factor = A[i][k] / A[k][k];
16         for(int j = k; j < n; j++) {
17             A[i][j] -= factor * A[k][j];
18         }
19         B[i] -= factor * B[k];
20     }
21     if (start >= n) {
22         break;
23     }
24 }
25
26 pthread_exit(NULL);
27 return NULL;
28 }

```

2.2.4 静态分配线程

对比于动态分配线程的主进程，此处不在主进程中对 k 进行迭代，仅创建一次线程，由各线程任务独自对 k 进行迭代，此处初始化一个 barrier 来同步各线程进度。

```

1 // 静态分配线程
2 void static_thread(void* (*task)(void*)) {
3     vector<pthread_t> thread_handles(thread_num);
4     vector<threadParam_t> params(thread_num);
5     pthread_mutex_init(&mutex_task, NULL);
6     pthread_barrier_init(&barrier, NULL, thread_num); // 初始化屏障，指定需
    ↳ 要等待的线程数为 thread_num
7     for (int t_id = 0; t_id < thread_num; t_id++) {
8         params[t_id].t_id = t_id;
9     }
10    for (int t_id = 0; t_id < thread_num; t_id++) {
11        pthread_create(&thread_handles[t_id], NULL, task, &params[t_id]);
12    }
13    for (int t_id = 0; t_id < thread_num; t_id++) {
14        pthread_join(thread_handles[t_id], NULL);
15    }
16    pthread_mutex_destroy(&mutex_task);
17    pthread_barrier_destroy(&barrier);

```

```
18     }
```

2.2.5 静态任务（静态分配线程）

与动态分配线程的线任务相比，此处在线程任务中对 k 进行迭代，每一次迭代后自行计算静态分配的 `chunk_size`，在处理完自己负责的任务后等待 barrier，所有线程都到达后，即对该主元行 k 处理完成，共同进行下一轮迭代。

```
1  // 静态任务
2  void* static_thread_static_simple_task(void* param) {
3      threadParam_t* p = (threadParam_t*)param;
4      int t_id = p->t_id;
5      int start;
6
7      for (int k = 0; k < n; k++) {
8          chunk_size = (n - (k + 1)) / thread_num + 1;
9          start = k + t_id * chunk_size + 1;
10         for(int i = start; (i < start + chunk_size) && (i < n); i++) {
11             float factor = A[i][k] / A[k][k];
12             for(int j = k; j < n; j++) {
13                 A[i][j] -= factor * A[k][j];
14             }
15             B[i] -= factor * B[k];
16         }
17
18         pthread_barrier_wait(&barrier);    // 等所有线程都完成消元，再进入下一轮
19     }
20
21     pthread_exit(NULL);
22     return NULL;
23 }
```

2.2.6 动态任务（静态分配线程）

此函数为动态分配线程的动态任务与静态分配线程的静态任务的结合体，线程任务中对 k 进行迭代，互斥访问 `next_row` 获取自己要处理的行，等待 barrier 实现同步。也可调整 `chunk_size` 实现对粒度的控制。

```
1  // 动态任务
2  void* static_thread_dynamic_simple_task(void* param) {
3      threadParam_t* p = (threadParam_t*)param;
4      int t_id = p->t_id;
5      int start;
```

```
6
7     for (int k = 0; k < n; k++) {
8         if (t_id == 0)
9             next_row = 0;
10        pthread_barrier_wait(&barrier);    // 等待重置 next_arr
11        while (1) {
12            pthread_mutex_lock(&mutex_task);
13            start = k + next_row + 1;
14            next_row += chunk_size;
15            pthread_mutex_unlock(&mutex_task);
16            for(int i = start; (i < start + chunk_size) && (i < n); i++) {
17                float factor = A[i][k] / A[k][k];
18                for(int j = k; j < n; j++) {
19                    A[i][j] -= factor * A[k][j];
20                }
21                B[i] -= factor * B[k];
22            }
23            if (start >= n) {
24                break;
25            }
26        }
27        pthread_barrier_wait(&barrier);    // 等所有线程都完成消元，再进入下一轮
28    }
29
30    pthread_exit(NULL);
31    return NULL;
32 }
```

2.3 实验及结果分析

2.3.1 测试方法

首先对动态分配线程的各方法进行测试，采用一大一小两个数据规模，测试各方法在不同线程数下的运行情况；通过对比观察测试结果，再全面对各个数据规模进行测试；随后对各种粒度进行测试，并加入对静态分配线程的测试；最后综合采用前面测试得到的参数再次对不同数据规模进行全面测试。

2.3.2 结果分析

(1) 初步观察对比

首先对不同线程数下动态分配线程的各个方法进行测试，测试一个小数据规模和一个大数据规模两种情况，对比结合观察。

小规模数据下，串行 AVX 耗时仅 0.3ms 左右，远低于所有并行方案，其在图7(a)上呈现一条几乎与 x 轴重合的横线。而各并行方案耗时均在 9ms 以上，随线程数增加呈上升趋势（经多次测试，图7(a)中

两个峰值为系统误差), 且各方法区分不大。

推测是小数据量下, 线程创建、同步开销占比极高, 掩盖了并行计算的收益。

在大数据规模情况下, 并行方案的优势便显现了出来。各并行方案的折现呈先下降再上升的趋势, 在 6-8 线程时达到最优。以 20 为临界点, 在线程数不高时, 各方法耗时: 静态 AVX < 动态 AVX < 串行 AVX < 动态任务 < 静态任务; 在线程数变得更大时, 各方法耗时: 串行 AVX < 静态 AVX < 动态 AVX < 静态任务 < 动态任务, 动态任务耗时超过了静态任务。

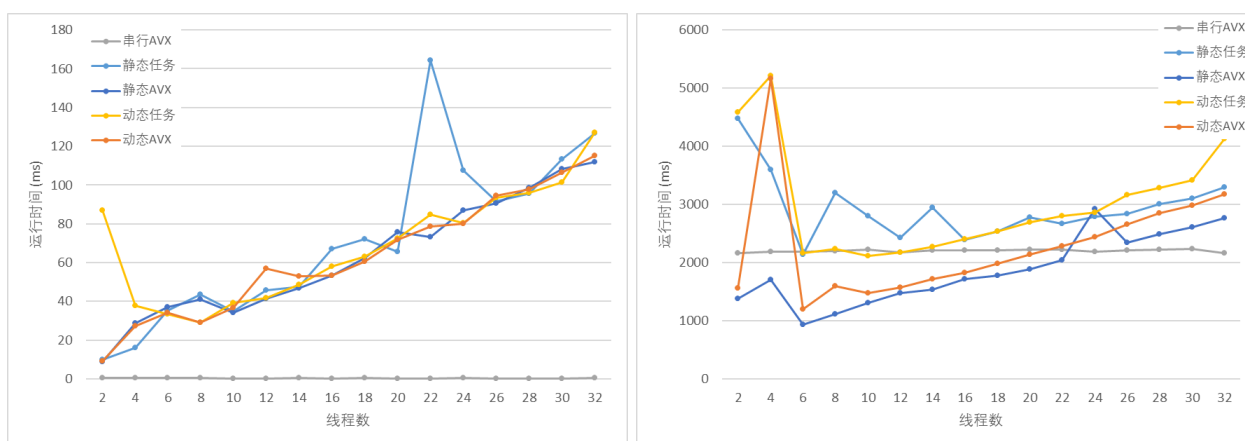
可见 AVX 编程带来的收益十分可观, 使串行计算的效率超过了普通的多线程方案。而从静态 AVX 始终优于动态 AVX, 以及在线程数较大时普通动态任务耗时超过了静态任务的情况看来, 线程数过多时, 线程创建、同步的开销以及上下文切换依旧会掩盖并行计算的收益。因此线程数不是越多越好, 在后续实验中采用一个较优的线程数 8 来进行测试。

线程数	串行 AVX	静态任务	静态 AVX	动态任务	动态 AVX
2	0.35559	9.7571	8.8836	86.915	9.0654
4	0.33752	16.115	28.767	37.869	27.198
6	0.32969	35.074	37.124	33.487	34.16
8	0.33683	43.7	40.913	28.931	29.031
10	0.30684	34.859	34.078	39.328	36.686
12	0.31678	45.775	41.444	41.889	56.768
14	0.35266	47.44	46.739	48.44	52.799
16	0.3073	67.052	53.257	57.946	53.443
18	0.32891	72.091	62.297	62.987	60.465
20	0.30897	65.524	75.65	72.3	71.794
22	0.30792	164.24	73.167	84.931	78.686
24	0.34141	107.62	87.008	80.571	80.134
26	0.31062	91.583	90.624	93.515	94.626
28	0.30893	95.509	98.437	96.081	97.819
30	0.30063	113.23	108.44	101.56	106.36
32	0.34997	126.77	112.03	127.11	115.06

表 4: 不同线程数下动态分配线程各算法运行时间 (数据规模: 128, 单位: ms)

线程数	串行 AVX	静态任务	静态 AVX	动态任务	动态 AVX
2	2165.5	4469.3	1381.1	4577.3	1559.2
4	2186.4	3599.1	1697.1	5216	5166.9
6	2187	2140	930.21	2164.6	1196.2
8	2199.3	3196.1	1108.6	2235.9	1594.8
10	2218.6	2795.7	1308.6	2114.6	1478.2
12	2173	2421	1470.7	2176.8	1574.6
14	2210.1	2940	1530	2263.6	1709.1
16	2208.8	2388.9	1716.8	2405.1	1819.3
18	2207.7	2529.2	1776.5	2535.4	1977
20	2224.2	2773.2	1879.2	2690.9	2131.7
22	2215.2	2664.2	2042.9	2803	2278
24	2180.2	2786.3	2920.2	2865.6	2433.5
26	2210.6	2830.1	2335.7	3159.4	2655.2
28	2217.8	3003.9	2489.3	3283.2	2851.5
30	2231.4	3101.8	2601.1	3413.9	2975.9
32	2158.9	3295.8	2763.6	4126.8	3167.1

表 5: 不同线程数下动态分配线程各算法运行时间 (数据规模: 2560, 单位: ms)



(a) 数据规模: 128

(b) 数据规模: 2560

图 2.7: 不同线程数下动态分配线程各算法运行时间

(2) 不同数据规模下各方法的效果 接下来测试在不同数据规模下, 各并行策略相对于串行策略的效率。

随着数据规模的增长, 各方法的运行时间都在增长, 但增长的速率明显不一致。最初几种并行策略均不如串行 AVX, 但很快, 静态 AVX 和动态 AVX 的耗时就来到了串行 AVX 之下, 且差距迅速变大。而虽然在测试数据中普通静态任务和动态任务并未优于串行 AVX, 但从图2.8中可以看出静态任务和动态任务的折线越来越贴近串行 AVX, 推测如果数据继续变大, 其也将会优于串行 AVX。

由此看来, 当数据规模越大时, 并行策略相对于串行策略的效果就越好。

规模	串行 AVX	静态任务	静态 AVX	动态任务	动态 AVX
128	0.30573	30.63	29.832	31.109	28.355
256	2.2851	84.71	61.252	63.676	69.753
384	7.7798	94.953	92.881	94.319	106.2
512	17.758	140.54	130.46	173.04	142.98
640	35.094	203.69	172.76	193.95	182.18
768	60.961	228.3	192.65	321.59	266.25
896	94.533	274.94	227.8	285.18	257.66
1024	144.06	396.75	271.53	317.24	278.66
1152	205.75	417.68	309.34	396.14	371.86
1280	280.67	490.39	418.86	489.56	418.12
1408	373.71	672.8	407.18	571.2	487.86
1536	484.08	750.37	439.24	685.73	611.3
1664	612.81	915.04	475.58	832.87	629.8
1792	759.74	1004	616.78	960.75	717.32
1920	933	1830.5	600.64	1098.5	932.52
2048	1143.7	1382.5	744.21	1438.3	1103.1
2176	1357.9	1838.9	794.98	1530	1035.3
2304	1620.4	1787.4	939.09	1690.7	1107.2
2432	1889.5	2021.6	1030.7	1904.5	1216
2560	2204.1	2497.3	1168.6	2240	1359.5

表 6: 不同数据规模下动态分配线程各算法运行时间 (线程数: 8, 单位: ms)

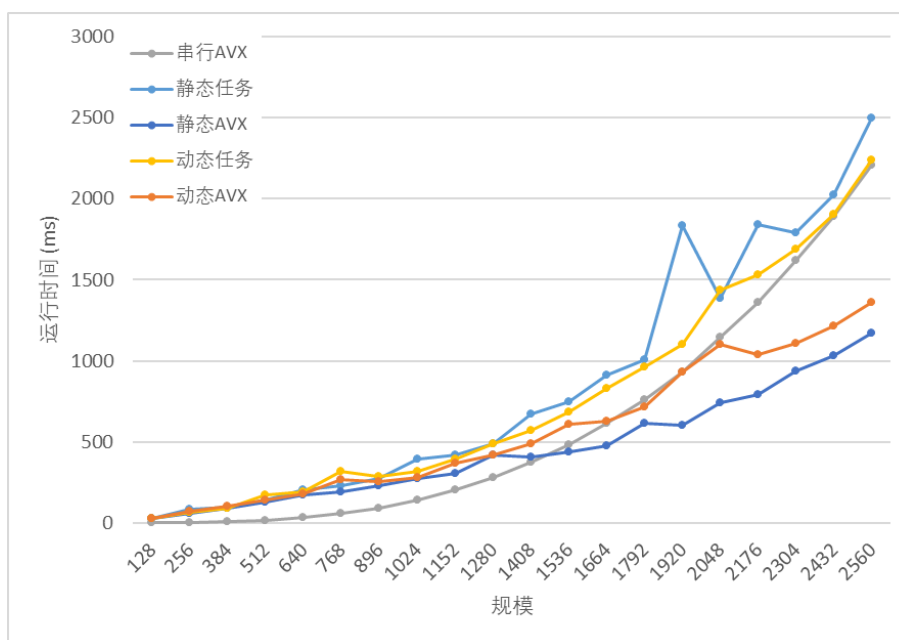


图 2.8: 不同数据规模下动态分配线程各算法运行时间

(3) 不同粒度下各方法的效果接下来再测试一下在不同粒度情况下, 动态分配任务的效果。采用一个较大的数据规模, 使得测试结果更加稳定, 同时并行策略的优势充分发挥, 不再与串行策略进行对比, 而是引入新的策略: 静态分配线程。

图2.9中暖色调的两条线是静态分配线程的数据，很明显，其远优于动态分配线程的数据，且折线十分平稳，运行稳定。静态分配线程的策略中，每一次主进程仅一次性创建多个线程，后续的迭代和同步通信由这些线程自己完成，相对于动态创建线程而言，其多了使用 barrier 同步通信的时间，但大大减少了反复创建线程的消耗。

再来分析粒度的影响，从图2.9中可以看出，粒度较小时，随着粒度的增大，动态分配进程的动态分配任务的运行时间逐渐变小，但在粒度为 8 时就达到稳定，后续基本不再变化，且与静态分配任务持平。考虑为锁竞争的开销与负载均衡的收益达到了平衡。

分配粒度	动态分配线程		静态分配线程	
	静态 AVX	动态 AVX	静态 AVX	动态 AVX
1	1250	1261.7	431.74	802.74
2	1268.1	1138.1	483.81	598.01
4	1185	1137.9	497.48	507.23
8	1154.7	1115.4	493.35	468.2
12	1154.4	1069.6	473.25	461.77
16	1213.9	1039.6	473.08	465.83
20	1174.7	1063.5	486.3	505.48
24	1364.1	1157.3	578.85	493.15
28	1272.7	1046.2	535.56	470.65
32	1389.8	1059.1	585.9	507.85
36	1176.4	1081.6	511.92	488.51
40	1177	1070.6	523.04	482.03
44	1533.4	1052.6	517.75	460.58
48	1165.1	1032.2	477.81	474.16
52	1162.9	1043.6	459.38	464.95
56	1158.3	1119.5	493.09	482.07
60	1259.9	1034.7	461.62	469.76
64	1159.9	1125.9	470.72	476.94

表 7: 不同分配粒度下各分配方式运行时间（数据规模：2560，线程数：8，单位：ms）

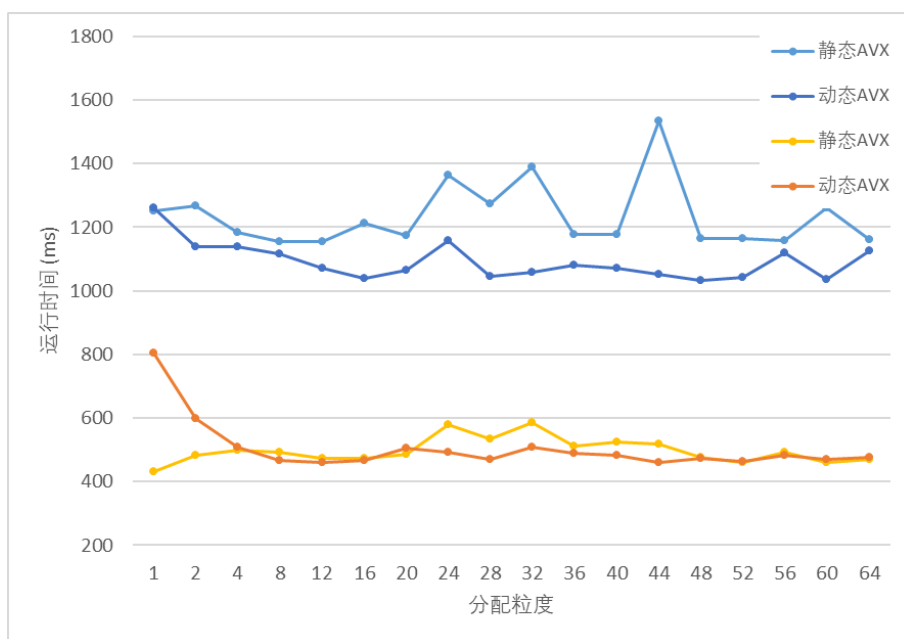


图 2.9: 不同分配粒度下各分配方式运行时间 (冷色动态分配线程-暖色静态分配线程)

(4) **综合测评**最后，选取一个较优的线程数和一个稳定优势的粒度来对不同数据规模进行测试。

通过图2.10可以看出，随着数据规模的增大，各策略的运行时间都在增长，但静态分配线程的两个策略增速较缓，且远优于动态分配线程，且优势越来越大。

因此通过本次实验得到的高斯消元法消元过程的最优策略为：线程数 6-8 + 静态分配线程 + 静态分配任务或粒度 >8 的动态分配任务 + AVX。

规模	动态分配线程		静态分配线程	
	静态 AVX	动态 AVX	静态 AVX	动态 AVX
128	35.371	29.715	1.4849	2.3377
256	58.787	58.844	3.492	4.9807
384	88.019	90.249	5.9728	8.9424
512	125.73	121.12	9.8593	12.752
640	155.45	161.97	15.357	19.231
768	182.44	198.7	22.977	25.12
896	248.12	252.75	33.378	35.28
1024	256.52	269.91	47.384	49.785
1152	316.67	317.01	61.064	68.453
1280	450.14	366.54	76.263	87.088
1408	459.42	437.61	99.58	108.56
1536	516.69	504.91	129.8	126.96
1664	577.66	511.71	159.09	158.56
1792	614.26	640.03	181.3	193.64
1920	729.76	625.47	239.19	236.85
2048	754.09	698.54	288.18	287.71
2176	821.43	832.85	353.38	332.38
2304	912.16	853.32	397.88	384.92
2432	989.93	928.18	477.99	441.2
2560	1120.2	1015.6	567.53	513.06

表 8: 不同数据规模下各分配方式运行时间 (分配粒度: 64, 线程数: 8, 单位: ms)

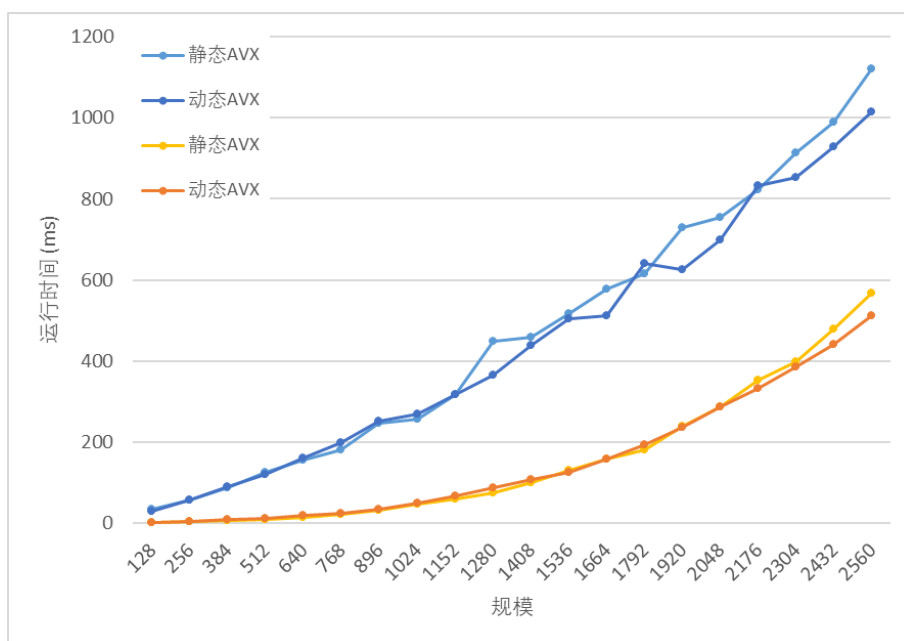


图 2.10: 不同数据规模下各分配方式运行时间 (冷色动态分配线程-暖色静态分配线程)

2.3.3 实际运行结果

动态线程 数据规模: 128
测试小数据规模在不同线程数情况, 初步观察各方法运行情况

线程数	串行AVX	静态任务	静态AVX	动态任务	动态AVX	原总用时
2	0.35559	9.7571	8.8836	86.915	9.0654	1149.8
4	0.33752	16.115	28.767	37.869	27.198	1102.9
6	0.32969	35.074	37.124	33.487	34.16	1401.8
8	0.33683	43.7	40.913	28.931	29.031	1429.1
10	0.30684	34.859	34.078	39.328	36.686	1452.6
12	0.31678	45.775	41.444	41.889	56.768	1861.9
14	0.35266	47.44	46.739	48.44	52.799	1957.7
16	0.3073	67.052	53.257	57.946	53.443	2320
18	0.32891	72.091	62.297	62.987	60.465	2581.7
20	0.30897	65.524	75.65	72.3	71.794	2855.8
22	0.30792	164.24	73.167	84.931	78.686	4013.3
24	0.34141	107.62	87.008	80.571	80.134	3556.7
26	0.31062	91.583	90.624	93.515	94.626	3706.6
28	0.30893	95.509	98.437	96.081	97.819	3881.5
30	0.30063	113.23	108.44	101.56	106.36	4298.9
32	0.34997	126.77	112.03	127.11	115.06	4813.2

图 2.11: 不同线程数下动态分配线程各算法运行时间 (小规模)

动态线程 数据规模: 2560
测试大规模数据在不同线程数下情况, 与小数据规模对比判断

线程数	串行AVX	静态任务	静态AVX	动态任务	动态AVX	原总用时
2	2165.5	4469.3	1381.1	4577.3	1559.2	1.4152e+05
4	2186.4	3599.1	1697.1	5216	5166.9	1.7866e+05
6	2187	2140	930.21	2164.6	1196.2	86180
8	2199.3	3196.1	1108.6	2235.9	1594.8	1.0335e+05
10	2218.6	2795.7	1308.6	2114.6	1478.2	99156
12	2173	2421	1470.7	2176.8	1574.6	98161
14	2210.1	2940	1530	2263.6	1709.1	1.0653e+05
16	2208.8	2388.9	1716.8	2405.1	1819.3	1.0539e+05
18	2207.7	2529.2	1776.5	2535.4	1977	1.1026e+05
20	2224.2	2773.2	1879.2	2690.9	2131.7	1.1699e+05
22	2215.2	2664.2	2042.9	2803	2278	1.2003e+05
24	2180.2	2786.3	2920.2	2865.6	2433.5	1.3186e+05
26	2210.6	2830.1	2335.7	3159.4	2655.2	1.3191e+05
28	2217.8	3003.9	2489.3	3283.2	2851.5	1.3846e+05
30	2231.4	3101.8	2601.1	3413.9	2975.9	1.4324e+05
32	2158.9	3295.8	2763.6	4126.8	3167.1	1.5512e+05

图 2.12: 不同线程数下动态分配线程各算法运行时间 (大规模)

动态线程 线程数量: 8
采用较优线程数量, 全面测试不同规模下情况

规模	串行AVX	静态任务	静态AVX	动态任务	动态AVX	原总用时
128	0.30573	30.63	29.832	31.109	28.355	1202.3
256	2.2851	84.71	61.252	63.676	69.753	2816.8
384	7.7798	94.953	92.881	94.319	106.2	3961.4
512	17.758	140.54	130.46	173.04	142.98	6047.8
640	35.094	203.69	172.76	193.95	182.18	7876.8
768	60.961	228.3	192.65	321.59	266.25	10698
896	94.533	274.94	227.8	285.18	257.66	11401
1024	144.06	396.75	271.53	317.24	278.66	14082
1152	205.75	417.68	309.34	396.14	371.86	17008
1280	280.67	490.39	418.86	489.56	418.12	20976
1408	373.71	672.8	407.18	571.2	487.86	25127
1536	484.08	750.37	439.24	685.73	611.3	29707
1664	612.81	915.04	475.58	832.87	629.8	34661
1792	759.74	1004	616.78	960.75	717.32	40586
1920	933	1830.5	600.64	1098.5	932.52	53952
2048	1143.7	1382.5	744.21	1438.3	1103.1	58118
2176	1357.9	1838.9	794.98	1530	1035.3	65570
2304	1620.4	1787.4	939.09	1690.7	1107.2	71448
2432	1889.5	2021.6	1030.7	1904.5	1216	80624
2560	2204.1	2497.3	1168.6	2240	1359.5	94695

图 2.13: 不同数据规模下动态分配线程各算法运行时间

动态线程（左两列） - 静态线程（右两列）			数据规模：2560		线程数量：8
使用并行策略显著优于串行策略的数据规模，测试不同粒度下动态分配任务情况					
分配粒度	静态AVX	动态AVX	静态AVX	动态AVX	原总用时
1	1250	1261.7	431.74	802.74	37462
2	1268.1	1138.1	483.81	598.01	34880
4	1185	1137.9	497.48	507.23	33276
8	1154.7	1115.4	493.35	468.2	32316
12	1154.4	1069.6	473.25	461.77	31590
16	1213.9	1039.6	473.08	465.83	31924
20	1174.7	1063.5	486.3	505.48	32299
24	1364.1	1157.3	578.85	493.15	35933
28	1272.7	1046.2	535.56	470.65	33251
32	1389.8	1059.1	585.9	507.85	35426
36	1176.4	1081.6	511.92	488.51	32584
40	1177	1070.6	523.04	482.03	32527
44	1533.4	1052.6	517.75	460.58	35644
48	1165.1	1032.2	477.81	474.16	31493
52	1162.9	1043.6	459.38	464.95	31308
56	1158.3	1119.5	493.09	482.07	32530
60	1259.9	1034.7	461.62	469.76	32260
64	1159.9	1125.9	470.72	476.94	32335

图 2.14: 不同分配粒度下各分配方式运行时间

动态线程 - 静态线程		分配粒度：64		线程数量：8	
使用较优粒度和较优线程数，比较静态分配线程和动态分配线程					
规模	静态AVX	动态AVX	静态AVX	动态AVX	原总用时
128	35.371	29.715	1.4849	2.3377	689.09
256	58.787	58.844	3.492	4.9807	1261
384	88.019	90.249	5.9728	8.9424	1931.8
512	125.73	121.12	9.8593	12.752	2694.6
640	155.45	161.97	15.357	19.231	3520.1
768	182.44	198.7	22.977	25.12	4292.3
896	248.12	252.75	33.378	35.28	5695.3
1024	256.52	269.91	47.384	49.785	6236
1152	316.67	317.01	61.064	68.453	7631.9
1280	450.14	366.54	76.263	87.088	9800.3
1408	459.42	437.61	99.58	108.56	11052
1536	516.69	504.91	129.8	126.96	12784
1664	577.66	511.71	159.09	158.56	14070
1792	614.26	640.03	181.3	193.64	16292
1920	729.76	625.47	239.19	236.85	18313
2048	754.09	698.54	288.18	287.71	20285
2176	821.43	832.85	353.38	332.38	23400
2304	912.16	853.32	397.88	384.92	25483
2432	989.93	928.18	477.99	441.2	28373
2560	1120.2	1015.6	567.53	513.06	32164

图 2.15: 不同数据规模下各分配方式运行时间

3 Barrier 功能多种实现与对比

3.1 问题描述

使用其他方式（如忙等待、互斥量、信号量等），自行实现不少于 2 种路障 Barrier 的功能，分析与 Pthread_barrier 相关接口功能的异同。提示：可采用课件上路障部分的案例，用其他 2 种方式实现相同功能；也可自行设定场景，实现 2 种或以上 barrier 的功能，并进行效率、功能等方面的展示比较。

3.2 算法设计与实现

使用条件变量 + 互斥量和信号量两种方式实现自定义屏障，并与原生屏障进行对比测试。

3.2.1 条件变量 + 互斥量

```
1  typedef struct {
2      pthread_mutex_t barrier_mutex;
3      pthread_cond_t barrier_cond;
4      int thread_count;
5      int arrived_count;
6      int cycle;
7  } CondMutexBarrier;
8
9  // 初始化屏障：条件变量 + 互斥量
10 void init_cond_mutex_barrier(CondMutexBarrier* cond_mutex_barrier, int
    ↪ thread_count) {
11     cond_mutex_barrier->thread_count = thread_count;
12     cond_mutex_barrier->arrived_count = 0;
13     cond_mutex_barrier->cycle = 0;
14     pthread_mutex_init(&cond_mutex_barrier->barrier_mutex, NULL);
15     pthread_cond_init(&cond_mutex_barrier->barrier_cond, NULL);
16 }
17
18 // 屏障：条件变量 + 互斥量
19 void cond_mutex_barrier_wait(CondMutexBarrier* cond_mutex_barrier) {
20     pthread_mutex_lock(&cond_mutex_barrier->barrier_mutex);
21     int current_cycle = cond_mutex_barrier->cycle;
22     cond_mutex_barrier->arrived_count++;
23
24     // 最后一个到达的线程：重置计数器并唤醒所有线程
25     if (cond_mutex_barrier->arrived_count == cond_mutex_barrier->thread_count) {
26         cond_mutex_barrier->arrived_count = 0;
27         cond_mutex_barrier->cycle++;
28         pthread_cond_broadcast(&cond_mutex_barrier->barrier_cond);
```



```

29     }
30     // 其他线程：阻塞等待，直到当前循环的所有线程到达
31     else {
32         while (current_cycle == cond_mutex_barrier->cycle) {
33             pthread_cond_wait(&cond_mutex_barrier->barrier_cond,
34                             ↪ &cond_mutex_barrier->barrier_mutex);
35         }
36     }
37     pthread_mutex_unlock(&cond_mutex_barrier->barrier_mutex);
38 }
39 // 销毁屏障：条件变量 + 互斥量
40 void destroy_cond_mutex_barrier(CondMutexBarrier* cond_mutex_barrier) {
41     pthread_mutex_destroy(&cond_mutex_barrier->barrier_mutex);
42     pthread_cond_destroy(&cond_mutex_barrier->barrier_cond);
43 }

```

3.2.2 信号量

```

1  typedef struct {
2      sem_t count_sem;
3      sem_t barrier_sem;
4      int thread_count;
5      int arrived_count;
6      pthread_mutex_t barrier_mutex;
7  } SemaphoreBarrier;
8
9  // 初始化屏障：信号量
10 void init_sem_barrier(SemaphoreBarrier* sem_barrier, int thread_count) {
11     sem_barrier->thread_count = thread_count;
12     sem_barrier->arrived_count = 0;
13     sem_init(&sem_barrier->count_sem, 0, thread_count);
14     sem_init(&sem_barrier->barrier_sem, 0, 0);
15     pthread_mutex_init(&sem_barrier->barrier_mutex, NULL);
16 }
17
18 // 屏障：信号量
19 void sem_barrier_wait(SemaphoreBarrier* sem_barrier) {
20     // 计数信号量 P 操作：记录线程到达（每到一个减 1）
21     sem_wait(&sem_barrier->count_sem);
22
23     pthread_mutex_lock(&sem_barrier->barrier_mutex);

```

```

24     sem_barrier->arrived_count++;
25     // 最后一个到达的线程：释放阻塞信号量（允许所有线程继续）
26     if (sem_barrier->arrived_count == sem_barrier->thread_count) {
27         sem_barrier->arrived_count = 0;
28         // 释放 thread_count 个信号量，唤醒所有等待线程
29         for (int i = 0; i < sem_barrier->thread_count; i++) {
30             sem_post(&sem_barrier->barrier_sem);
31         }
32     }
33     pthread_mutex_unlock(&sem_barrier->barrier_mutex);
34
35     // 阻塞信号量 P 操作：等待所有线程到达
36     sem_wait(&sem_barrier->barrier_sem);
37     // 支持重复使用：在通过屏障后，将 count_sem 加回来
38     sem_post(&sem_barrier->count_sem);
39 }
40
41 // 销毁屏障：信号量
42 void destroy_sem_barrier(SemaphoreBarrier* sem_barrier) {
43     sem_destroy(&sem_barrier->count_sem);
44     sem_destroy(&sem_barrier->barrier_sem);
45     pthread_mutex_destroy(&sem_barrier->barrier_mutex);
46 }

```

3.2.3 测试函数

barrier_main 作为创建线程的主进程，在其中初始化和销毁屏障，barrier_task 作为线程函数，在其中模拟使用屏障的逻辑。

```

1 // 测试任务
2 void* barrier_task(void* parm) {
3     threadParam_t* params = (threadParam_t*)parm;
4
5     long long something = 0;
6     for (int i = 0; i < 10000; i++) {
7         something += i;
8     } // 模拟执行其他逻辑
9
10    if (params->barrier_type == "std_barrier") {
11        pthread_barrier_wait((pthread_barrier_t*)params->barrier_instance);
12    }
13    else if (params->barrier_type == "cond_mutex_barrier") {

```

```
14         cond_mutex_barrier_wait((CondMutexBarrier*)params->barrier_instance);
15     }
16     else if (params->barrier_type == "sem_barrier") {
17         sem_barrier_wait((SemaphoreBarrier*)params->barrier_instance);
18     }
19
20     something = 0;
21     for (int i = 0; i < 10000; i++) {
22         something += i;
23     }
24
25     pthread_exit(NULL);
26     return NULL;
27 }
28
29 // 测试任务主函数
30 void barrier_main(string barrier_type, int thread_count) {
31     vector<pthread_t> thread_handles(thread_count);
32     vector<threadParam_t> params(thread_count);
33
34     if (barrier_type == "std_barrier") {
35         pthread_barrier_init(&barrier, NULL, thread_count);
36     }
37     else if (barrier_type == "cond_mutex_barrier") {
38         init_cond_mutex_barrier(&cond_mutex_barrier, thread_count);
39     }
40     else if (barrier_type == "sem_barrier") {
41         init_sem_barrier(&sem_barrier, thread_count);
42     }
43
44     for (int i = 0; i < thread_count; i++) {
45         params[i].t_id = i;
46         params[i].barrier_type = barrier_type;
47         if (barrier_type == "std_barrier") {
48             params[i].barrier_instance = &barrier;
49         }
50         else if (barrier_type == "cond_mutex_barrier") {
51             params[i].barrier_instance = &cond_mutex_barrier;
52         }
53         else if (barrier_type == "sem_barrier") {
54             params[i].barrier_instance = &sem_barrier;
55         }
56     }
```

```
56     pthread_create(&thread_handles[i], NULL, barrier_task,  
57     ↪     (void*)&params[i]);  
58 }  
59 for (int i = 0; i < thread_count; i++) {  
60     pthread_join(thread_handles[i], NULL);  
61 }  
62 if (barrier_type == "std_barrier") {  
63     pthread_barrier_destroy(&barrier);  
64 }  
65 else if (barrier_type == "cond_mutex_barrier") {  
66     destroy_cond_mutex_barrier(&cond_mutex_barrier);  
67 }  
68 else if (barrier_type == "sem_barrier") {  
69     destroy_sem_barrier(&sem_barrier);  
70 }  
71 }
```

3.3 实验及结果分析

3.3.1 测试方法

测试在不同的线程数下，执行包含屏障的模拟任务的线程的运行时间。

3.3.2 结果分析

从图3.16中可以看出三种方法几乎无差异，运行时间都随着线程数增大而增大，在 26 线程时数据陡增，多次测试均出现此情况，目前不知道是何原因。

线程数	标准屏障	条件变量 + 互斥锁	信号量
2	0.15204	0.14956	0.10805
4	0.19512	0.19167	0.2923
6	0.26442	0.22942	0.2443
8	0.36698	0.28981	0.26833
10	0.36438	0.34277	0.35338
12	0.4262	0.43865	0.38629
14	0.46392	0.45603	0.4421
16	0.53304	0.51148	0.54748
18	0.58705	0.54866	0.54521
20	0.60475	0.63539	0.62358
22	0.69949	0.69214	0.68926
24	0.75933	0.74362	0.71235
26	1.1954	1.1831	1.0611
28	1.3558	1.3006	1.2911
30	1.4613	1.4833	1.4985
32	1.5131	1.6089	1.5994

表 9: 不同线程数下三种同步机制的运行时间 (单位: ms)

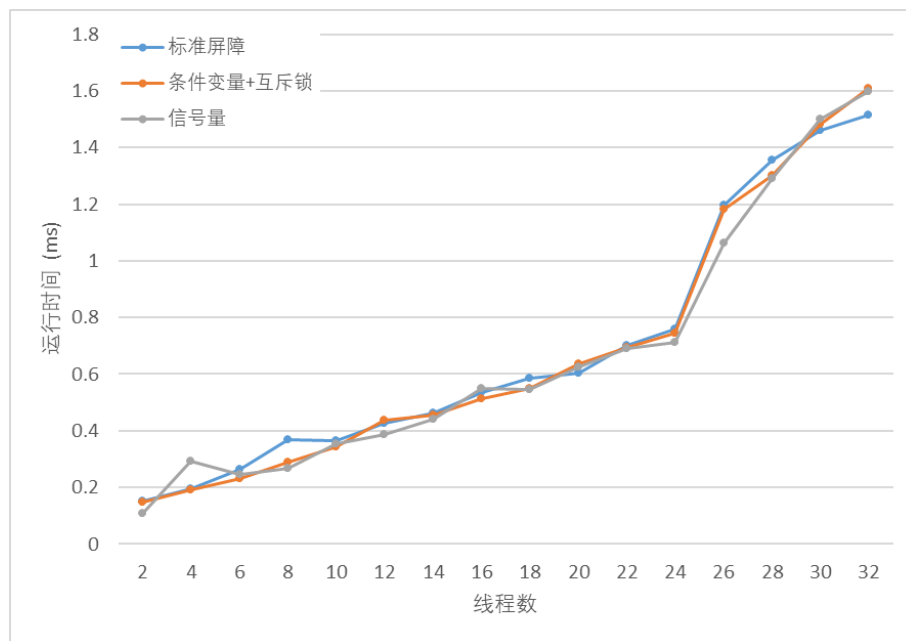


图 3.16: 不同线程数下三种同步机制的运行时间

3.3.3 实际运行结果

线程数	标准屏障	条件变量+互斥锁	信号量	原总用时
2	0.15204	0.14956	0.10805	4.0965
4	0.19512	0.19167	0.2923	6.7909
6	0.26442	0.22942	0.2443	7.3814
8	0.36698	0.28981	0.26833	9.2512
10	0.36438	0.34277	0.35338	10.605
12	0.4262	0.43865	0.38629	12.511
14	0.46392	0.45603	0.4421	13.62
16	0.53304	0.51148	0.54748	15.92
18	0.58705	0.54866	0.54521	16.809
20	0.60475	0.63539	0.62358	18.637
22	0.69949	0.69214	0.68926	20.809
24	0.75933	0.74362	0.71235	22.153
26	1.1954	1.1831	1.0611	34.395
28	1.3558	1.3006	1.2911	39.476
30	1.4613	1.4833	1.4985	44.431
32	1.5131	1.6089	1.5994	47.214

图 3.17: 不同线程数下三种同步机制的运行时间